

```
"""
```

```
Epic 2: Visualizing and Analysing the Data
```

```
Project: Rising Waters - Flood Prediction
```

```
-----
```

```
Covers all Epic 2 sub-tasks:
```

1. Importing the Libraries
2. Reading the Dataset
3. Univariate Analysis
4. Multivariate Analysis
5. Descriptive Analysis

```
Requirements:
```

```
pip install pandas numpy matplotlib seaborn openpyxl
```

```
Usage:
```

```
python epic2_visualizing_and_analysing.py --data data/rainfall_dataset.csv
```

```
(or point --data at the .xlsx file downloaded in Epic 1)
```

```
"""
```

```
import os
```

```
import argparse
```

```
# -----
```

```
# 1. Importing the Libraries
```

```
# -----
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
sns.set(style="whitegrid")
```

```
# -----
```

```
# 2. Reading the Dataset
```

```
# -----
```

```
def read_dataset(file_path):
```

```
    """Read the dataset from a CSV or Excel file into a DataFrame."""
```

```
    if not os.path.exists(file_path):
```

```
        raise FileNotFoundError(f"Could not find dataset at: {file_path}")
```

```
    if file_path.lower().endswith(".xlsx"):
```

```
        df = pd.read_excel(file_path)
```

```
    elif file_path.lower().endswith(".csv"):
```

```

        df = pd.read_csv(file_path)
    else:
        raise ValueError("Unsupported file format. Use .csv or .xlsx")

    print(f"Dataset loaded: {df.shape[0]} rows, {df.shape[1]} columns")
    return df

# -----
# 3. Univariate Analysis
# -----
def univariate_analysis(df, output_dir):
    """Plot distribution of each numeric feature individually."""
    numeric_cols = df.select_dtypes(include=np.number).columns

    for col in numeric_cols:
        plt.figure(figsize=(6, 4))
        sns.histplot(df[col].dropna(), kde=True, color="teal")
        plt.title(f"Distribution of {col}")
        plt.xlabel(col)
        plt.ylabel("Frequency")
        plt.tight_layout()

        save_path = os.path.join(output_dir, f"univariate_{col}.png")
        plt.savefig(save_path)
        plt.close()
        print(f"Saved univariate plot: {save_path}")

# -----
# 4. Multivariate Analysis
# -----
def multivariate_analysis(df, output_dir):
    """Plot a correlation heat map across all numeric features."""
    fig = plt.gcf()
    fig.set_size_inches(15, 15)

    sns.heatmap(
        df.corr(numeric_only=True),
        annot=True,
        cmap="summer",
        linewidths=1,
        linecolor="k",
        square=True,
        vmin=-1, vmax=1,
        cbar_kws={"orientation": "vertical"},
    )

```

```

plt.title("Multivariate Analysis - Correlation Heat Map")
plt.tight_layout()

save_path = os.path.join(output_dir, "multivariate_heatmap.png")
plt.savefig(save_path)
plt.close()
print(f"Saved multivariate heat map: {save_path}")

# -----
# 5. Descriptive Analysis
# -----
def descriptive_analysis(df, output_dir):
    """Generate and save descriptive statistics of the dataset."""
    summary = df.describe(include="all").transpose()
    print("\n--- Descriptive Statistics ---")
    print(summary)

    save_path = os.path.join(output_dir, "descriptive_statistics.csv")
    summary.to_csv(save_path)
    print(f"Saved descriptive statistics: {save_path}")

def main():
    parser = argparse.ArgumentParser(description="Epic 2: Visualizing and Analysis")
    parser.add_argument(
        "--data",
        default="data/rainfall_dataset.csv",
        help="Path to the dataset file (.csv or .xlsx)",
    )
    parser.add_argument(
        "--output",
        default="outputs/eda",
        help="Directory to save plots and statistics",
    )
    args = parser.parse_args()

    os.makedirs(args.output, exist_ok=True)

    df = read_dataset(args.data)
    univariate_analysis(df, args.output)
    multivariate_analysis(df, args.output)
    descriptive_analysis(df, args.output)

    print("\nEpic 2 analysis complete. Results saved in:", args.output)

```

```
if __name__ == "__main__":  
    main()
```